Worst case and what triggers it
My adjacency-matrix version of Stoer–Wagner does $\Theta(n^3)$ work no matter how the graph is structured. Each of the n − 1 phases scans whole rows of the matrix whether those entries hold edges or not. For operation count, then, every graph of size n is the worst case. What surprised me was that the constant factor does depend on the input. In random graphs the vertices in A are scattered across the index range, so the CPU can't predict the inA[v] branch well. At n = 100, random_dense took 0.95 ms and the cycle took 0.42 ms. That is 2.2× slower for the same number of operations, so in practice random, unstructured graphs are the real worst case. Memory is the actual limit on size. The n × n matrix of long long takes $8n^2$ bytes, which is about 20 GB at n = 50,000. Weights are also summed in long long, so a graph with total weight above about 9.2×10^{18} would overflow.

Hardest design decision
The subtlest line in my code reads the cut of the phase from w[last]. That value is only correct because the update loop skips vertices already in A. As a result, w[last] still holds last's total connection to the vertices added before it. I didn't see that this line depended on a condition in a different loop until I had an AI tool review my code. It pointed out that if the update loop ever stopped skipping A, w[last] would quietly give the wrong cut without any error. I worked through why the invariant holds and added a comment next to the line explaining it, so a later change wouldn't break it without anyone noticing. What I took from this is that some code is correct only because of something that happens elsewhere, and those dependencies need to be written down.

What I'd do differently
Initially, I would maintain an array representing only those vertices that have not yet been incorporated into set A. I would remove each such vertex from this array via a call to swap/remove as it becomes part of A. This would eliminate the dependency on the inA branch that resulted in a 2.2X degradation in performance, and would also reduce the amount of data that needs to be scanned by roughly half. Next, I would convert my representation of the matrix from using vector> to a single flat n*n array. This would result in only one memory allocation and would preclude the use of row pointers for indexing. Finally, I would implement the heap-based version of Stoer-Wagner with a cost of O(nm+n^2log(n)) as my matrix-based version was defeated by Karger on sparse graphs which directly conflicted with my expectations. Lastly, I would measure the performance of the selection-loop and update-loop individually to allow for verification of prediction P8.

What I now understand
I now fully comprehend why Stoer Wagner is correct. The minimum cut found during a particular iteration is between two vertices s and t that have just been merged. There are two possible scenarios: either the global min-cut splits s and t, in which case this iteration produces it; or s and t are on the same side of the global min-cut, in which case merging them cannot destroy it. Regardless of which scenario occurs, after processing all phases of the algorithm, we have found the global min-cut. In addition, I now appreciate why having the same number of

operations does not equate to having equal run-times. My results indicate that software-related branch prediction impacts may rival those related to computational complexity theory.